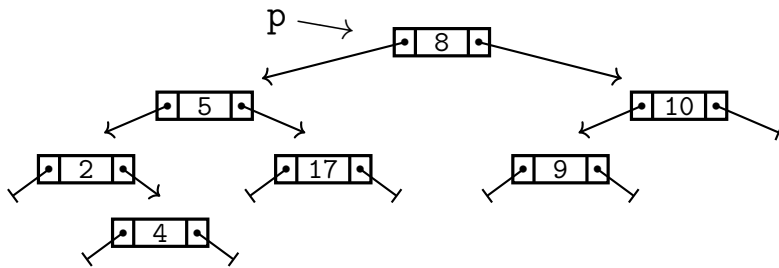


Balanceamento

Considere um ponteiro `p` para uma árvore binária cujos nós armazenam chaves inteiras, conforme ilustrado na Figura abaixo.

Por exemplo,



A menor altura possível dentre todas as árvores binárias com N nós, denotada por $h_{\min}(N)$, é dada por:

$$h_{\min}(N) = \lceil \log_2(N + 1) \rceil - 1.$$

Essa expressão pode ser calculada da seguinte maneira:

```
int hmin(int N){
    int h = 0;
    N++;
    while(N>0){
        N = N/2;
        h++;
    }
    return h-1;
}
```

Com base nessa definição, classificamos o balanceamento de uma árvore binária da seguinte forma:

- A árvore binária T é dita **fortemente balanceada** se sua altura satisfaz

$$h(T) = h_{\min}(N).$$

- A árvore binária T é dita **fracamente balanceada** se

$$h(T) \leq h_{\min}(N) + c,$$

onde c é uma constante pequena, usualmente $c = 1$. Essa definição admite uma pequena tolerância em relação à altura mínima, mantendo ainda um bom desempenho assintótico.

- A árvore binária T é dita **super fracamente balanceada** se

$$h(T) \leq 2 h_{\min}(N).$$

Essa condição impõe apenas um limite superior global para a altura da árvore, garantindo que ela não seja excessivamente degenerada.

Caso a árvore não satisfaça nenhuma das condições acima, ela é considerada **não balanceada**.

Tarefa.

Implemente um algoritmo que classifique o balanceamento da árvore binária apontada por `p` e armazene em `resp` um dos seguintes valores:

- 1, se a árvore for fortemente balanceada;
- 2, se a árvore for fracamente balanceada;
- 3, se a árvore for super fracamente balanceada;
- 4, se a árvore não for balanceada.

No exemplo apresentado, o valor armazenado em `resp` deve ser:

$$\text{resp} \Rightarrow 1.$$